

AI Report

We are moderately confident this text is

AI Generated

AI Probability

60%

This number is the probability that the document is AI generated, not a percentage of AI text in the document.

Plagiarism



The plagiarism scan was not run for this document. Go to gptzero.me to check for plagiarism.

Ephemeral Interfaces: Design Patterns for Task-Scoped Generative UI Components Daniel Kumlin IE Univ - 4/8/2026

Malena Willi

Ephemeral Interfaces: Design Patterns for Task-Scoped Generative UI Components Daniel Kumlin IE University
Capstone Project Supervisor: Prof. Jose Antonio Garcia Escand' on Date (April 2026) 1

Acknowledgements

I would like to thank everyone who has helped guide me through the process of creating this thesis.
It was a personal idea I really wanted to explore and felt empowered to try this with all the positive feedback I got from friends and colleagues.

I would like to thank my supervisor, Prof. Jose Antonio Garcia Escand' on, for his guidance, feedback, and support throughout this capstone project.

I am also very grateful to the participants who conducted the study.
It was an interesting twist to traditional surveys so I really appreciate everyone who took the time to take it.

Finally thank you for my family and friends who have supported me along the way.

Let the capstone begin.

Abstract

Generative UI has become a broader topic in software design because it promises in- terfaces that adapt to user goals instead of remaining static.

Yet this promise collides with modern software use, where users still expect stable experiences, predictable controls, and durable mental models.

This thesis investigates whether ephemeral, task-scoped generative UI components can offer a bounded alternative to full interface regeneration by appearing only when locally useful and disappearing without destabilising the host application.

The thesis develops and tests a framework for ephemeral support through a web-based research artifact. The main individual contributions are the framework operationalisation, a constrained component-specification model for temporary support, a validation pipeline for generated interface specifications, a bounded ephemeral overlay approach that preserves interface continuity, and an interactive survey artifact for empirical evaluation. The artifact was tested in a within-subjects user study comparing a baseline interface with an ephemeral condition on a bounded slide-reordering task.

The results show a mixed outcome.

The ephemeral condition showed a descriptively higher task completion rate and significantly higher helpfulness ratings, while perceived control remained stable, but it did not improve completion time or overall preference and significantly increased interaction count and perceived intrusiveness.

These findings suggest that ephemeral generative support is most credible not as a general replacement for existing interfaces, but as a bounded design strategy for bounded, low-consequence sub-tasks within broader workflows, especially where lightweight contextual guidance may be useful and interruption costs remain manageable.

The thesis therefore contributes a framework, design guidelines, and an empirical basis for assessing when temporary generative assistance can add value without disrupting the user experience.

Keywords: generative AI, design, UI/UX, web apps, ephemeral interfaces, human-computer interaction, productivity software, design science research

1 Introduction

1.1 Problem and Motivation

Large language models are no longer confined to generating textual content; they are increasingly being used to produce user interfaces that adapt to the goals and context of individual users.

This shift has opened a design space in which the structure of an application can respond to what a user needs rather than remaining fixed by its developers.

Yet the promise of adaptive, AI-generated interfaces introduces a fundamental tension.

On the one hand, static interfaces constrain users to predetermined layouts and user journeys that may not match the diversity of real-world tasks.

On the other hand, fully generative approaches that replace the entire interface risk undermining the predictability and continuity on which productive use depends.

This tension is especially acute in productivity software, where users build durable mental models of how their tools behave.

When an interface is regenerated anew in response to each new request, orientation is lost so users are required to relearn the interface, how to use it, and where key controls are located.

The design challenge, therefore, is not simply whether AI can generate useful interfaces, but how generative capability can be introduced into an existing application without eroding the stability that makes it usable in the first place.

A promising middle ground lies in ephemerality : rather than replacing an interface entirely, the system could surface temporary, task-scoped components that appear only when contextually justified and disappear once their purpose has been served.

Such components would augment the host application at the point of need while leaving the surrounding interface intact.

Although this idea draws on established principles from both generative UI research and interaction design theory, a coherent framework that specifies when ephemeral components should appear, how their scope should be bounded, and how they should be withdrawn without disrupting the user's workflow has not been articulated in prior design work.

1.2 Literature Review

While LLMs have been primarily used to generate content through text, they are becoming more prevalent in the generation of user interfaces (UI).

It is a way to move beyond static interfaces by generating structures based on an underlying set of primitives tailored to the users' goals.

Prior work suggests that when working with LLMs, users prefer to express themselves through UI instead of a plain chat.

This is because the system can present its functionality properly instead of forcing it through text (Ahmed & Imran, 2025; Leviathan et al., 2025; Tang et al., 2025).

However, most of the evidence comes from fully generated, end-to-end user interfaces, which differs from the design challenge addressed in this thesis, which is to augment existing applications where users expect a certain degree of stability and predictability in their tools.

This raises the question of how an underlying set of core primitive UI components can be integrated into already established workflows instead of replacing the entire state of the interface with AI.

The problem with generative UI interfaces is that while they can increase the relevance of the task at hand, they can threaten predictability and user control.

In real-world domains such as banking, their rigid interfaces were built on a set of predefined rules that fail to meet the ever-changing needs of their users across services (Tambi, 2020).

Approaches that dynamically generate interfaces can address the problem of not tailoring just to the content needed to serve the user, but the actual structure of the interaction over time.

Yet, these new systems raise an unresolved question: when generative UI is beneficial, how can it be introduced to users without it corroding the user's mental model of how the application should work?

This implies that the design pattern put in place for generative UI must manage the tradeoff between adaptivity and consistency (Bieniek et al., 2024).

To actually manage the tradeoff of leveraging generative UI, instead of having the entire interface be generated with AI, we could restrict it to ephemeral, so that task-scoped components appear only when needed and disappear when their use case has passed.

Past work on ephemeral interfaces argues that the permanence in ubiquitous environments can contribute to cognitive overload as there are too many controls and artifacts present (Döring et al., 2013).

Conversely, ephemerality can reduce the persistent clutter of artifacts and help with better interactions for the user within a specific time frame and context.

While this research was grounded in physical interaction design, it provides principles that can be transferred to digital systems.

These ephemeral elements should be discoverable when needed, easy to dismiss, and designed to disappear without affecting the surrounding environment's knowledge.

For generative UI, it suggests that ephemeral components could be a natural fit for LLMs as the system could generate the UI contextually in almost real time (Bieniek et al., 2024).

However, it must be kept in mind how the component lifecycle should be carefully designed to prevent an unstable user experience.

Ephemeral generative components have been implemented in past research without a full interface replacement, but with a temporary UI as an intermediate layer for exploration and understanding.

For agentic programming, research used ephemeral interfaces that sat between the prompt and code generation to improve observability and comprehension of LLM generated code.

It helped developers steer their results without needing to inspect the raw code (Cheng et al., 2024).

This shows how ephemeral UI can function as a way to help users with a workflow that lets them steer the outcome and iteratively refine stochastic tasks for exploring alternatives before taking concrete actions.

However, this application of ephemeral UI was done in a narrow domain and intent.

This leaves the question of how ephemeral patterns can generalise to everyday productivity applications where tasks are diverse.

If not implemented properly it could result in a disruption of interruptions, context switching, or inconsistency of familiar workflow.

Taking all existing literature into account, it establishes three relevant and incomplete variables to this capstone.

Firstly, fully generative UI experiences can improve the user experience and enjoyment of interacting with generative AI applications compared to only chat-only interaction (Ahmed & Imran, 2025; Leviathan et al., 2025; Tang et al., 2025).

Secondly, deployments of dynamic interface generations in specific domains can address the shortcomings of problems in need of rule-based logic in which needs are constantly changing and require tailoring (Tambi, 2020).

Thirdly, research on ephemerality shows how temporary time-bound interaction artifacts can reduce cognitive overload by decreasing the amount of permanent UI

accumulation for the user.

Nonetheless, a clear gap remains at the intersection of limited guidance on how to design ephemeral, task-scoped generative UI components that can improve existing applications while preserving a predictable user experience.

Prior work does not articulate reproducible design patterns that define when generative components should appear, how their scope should be bounded with an exit that does not disrupt the users' mental model of the application, and how they should preserve a predictable user interface.

This capstone addresses that gap by generalising ephemerality into a framework of component patterns to integrate into everyday applications.

1.3 Research Question and Contribution

The gap identified in the literature motivates the following research question:

How can ephemeral, task-scoped generative UI components be designed to improve existing applications without disrupting the user experience?

To address this question, the thesis makes a set of connected contributions.

First, it synthesises a conceptual framework that defines a taxonomy of ephemeral support roles, a four-stage component lifecycle, and a set of design principles that constrain how temporary generative components may be introduced into an otherwise stable interface.

Second, it operationalises that framework through a constrained component-specification model that defines what kinds of temporary support may be generated and how they may be structured.

Third, it implements a validation pipeline and bounded overlay approach that checks generated specifications against structural and scenario constraints before rendering them as temporary, dismissible interface layers that preserve continuity with the host application.

Fourth, it builds an interactive survey artifact: an experimental website that implements both a conventional baseline interface and an ephemeral condition governed by the framework's rules, with event logging and questionnaire capture built into the same system.

Fifth, it evaluates the artifact through a comparative within-subjects user study that measures task completion, efficiency, interaction effort, perceived helpfulness, intrusiveness, and user control across both conditions.

Together, these contributions move beyond the existing literature by offering a concrete operationalisation and empirical evidence of how ephemeral generative UI performs in a bounded productivity scenario.

1.4 Thesis Outline

The remainder of this thesis is organised as follows.

Section 2 presents the research methodology, including the conceptual framework, the design of the artifact, and the evaluation plan.

Section 3 describes the technical implementation of the experimental website and its support-generation pipeline.

Section 4 reports the results of the user study across all behavioural and self-reported measures.

Section 5 interprets the findings in relation to prior work and discusses the limitations and robustness of the evaluation.

Section 6 outlines directions for future work, and Section 7 concludes.

2 Methodology

2.1 Research Design

This project adopts a Design Science Research (DSR) approach (Delpont et al., 2024) because its primary goal is not only to study an existing phenomenon, but to create and evaluate an artifact that addresses an identified design problem.

The problem established in the literature is that current generative interfaces often fall at one of two extremes: either assistance is constrained to static interface structures, or entire experiences are dynamically generated in ways that can undermine predictability and continuity.

In response, this project proposes a conceptual framework for ephemeral, task-scoped generative UI components intended to augment existing applications without disrupting user workflow.

The methodological contribution therefore lies in developing a framework that translates ideas from prior work on ephemerality, generative UI, and interaction support into a form that can guide design decisions in a broader productivity context.

Following this DSR logic, the study proceeds in three connected stages.

First, a conceptual framework is constructed from existing literature in order to define the taxonomy, lifecycle, and design principles of ephemeral interface components.

Second, that framework is instantiated in the artifact as a web-based research instrument, allowing the proposed ideas to be embodied in a controlled interactive environment.

Third, the artifact is evaluated through a comparative user study that contrasts an original interface with an ephemeral condition on a single, bounded task.

Hence, the evaluation is not detached from theory.

The framework informs the design of the artifact, and the artifact in turn provides the basis for assessing whether ephemeral augmentation can improve task performance and user experience without introducing excessive intrusiveness.

2.2 Conceptual Framework

This study develops a conceptual framework for ephemeral, task-scoped generative UI components to guide both artifact design and evaluation.

In one sentence, the framework specifies

which temporary support roles may appear, how those supports are triggered and withdrawn, and which constraints keep them useful without destabilising the host interface.

Positioning the framework here in the methodology is important because it defines the concepts that the artifact operationalises and the criteria against which the two interface conditions are later compared.

The first element is a taxonomy of component roles.

Rather than classifying components by visual form, such as whether they appear as a panel, popover, or inline element, the taxonomy classifies them by the function they serve within the workflow.

Three roles are distinguished.

Interpretive support helps users understand context, available options, outputs, or likely consequences before acting.

Refinement support helps users steer or configure an intended action before commitment, for example by comparing alternatives or adjusting parameters.

Task-execution support helps users complete a bounded subtask inside the surrounding application without replacing the broader workflow.

Framing these roles as a taxonomy is methodologically useful because it makes explicit what kind of assistance is being introduced in each scenario and avoids treating ephemeral behaviour as a single undifferentiated interface style.

Ephemeral, task-scoped generative support Task-execution Interpretive support Refinement support support
Clarifies context Compares or adjusts Supports a bounded and consequences options subtask

Figure 1: Taxonomy of ephemeral, task-scoped generative support roles.

The second element is a lifecycle describing how an ephemeral component enters and leaves use. Alternative stage patterns can be found in the literature, ranging from the minimal logic of temporary appearance and disappearance (Döring et al., 2013) to more scaffolded models in which users inspect and steer an intermediate layer before committing to an outcome (Cheng et al., 2024). For this study, the most appropriate synthesis is a four-stage lifecycle of trigger, instantiation, interaction, and dismissal.

The component is triggered when a clear user goal or contextual need is detected that is not adequately supported by the persistent interface alone.

That trigger may be explicit, such as a user request for help, or inferred from local behavioural and contextual signals, but only when the resulting support remains low-risk,

relevant, and easy to dismiss.

The component is then instantiated as a temporary interface element embedded within the existing application rather than replacing the surrounding interface.

During interaction, users inspect, steer, or complete the immediate subtask while remaining oriented within the primary workflow.

Finally, the component is dismissed once the task is completed, abandoned, or no longer contextually relevant. This lifecycle was selected because it retains the conceptual clarity of ephemerality while remaining concrete enough to implement as a state-based interaction model in the artifact.

Trigger Ephemeral component Dismissal Instantiation lifecycle Interaction

Figure 2: Lifecycle of an ephemeral, task-scoped generative UI component: trigger, instantiation, interaction, and dismissal.

The third element is a set of design principles that constrain how ephemeral components should be introduced into an otherwise stable interface.

Five principles guide the framework.

Contextual relevance means support should appear only when it serves a clear task-related purpose in the current situation.

Bounded scope means support should remain limited to the immediate subtask rather than taking over the larger workflow.

Interface continuity means the temporary component should preserve the surrounding page structure and the user's sense of place, rather than redirecting the interface toward an unrelated experience.

User control means users should be able to inspect, refine, confirm, or dismiss the support rather than being forced

into a system decision.

Graceful disappearance means the component should recede cleanly once it is no longer needed.

When support is inferred rather than explicitly invoked, these principles impose an additional safety constraint for inferred assistance, where it may suggest or preload options but should not perform destructive or strongly consequential actions without explicit user confirmation.

Together, these principles translate the theoretical idea of ephemerality into practical design criteria that can be applied consistently during artifact development and later assessed in the evaluation.

2.3 Artifact Development

The artifact developed in this project is a website used as a research instrument for evaluating the proposed framework in a controlled but realistic interaction setting.

Rather than building a fully autonomous or fully generative application, the website is designed to preserve a recognisable baseline interface while allowing ephemeral, task-scoped components to be introduced at selected moments.

This makes it possible to test whether the framework can augment the existing user journey without replacing the primary workflow altogether.

To operationalise the framework, the website will implement two interface conditions: an original condition using a well-known, conventional interface and an ephemeral condition in which temporary, task-scoped components are introduced according to the taxonomy, lifecycle, and design principles defined above.

Both conditions use the same underlying slide-deck interface, task mechanics, and submission logic.

The original condition presents the task without temporary support, whereas the ephemeral condition will surface temporary interface elements only when additional guidance, refinement, or task support is needed.

In this way, the comparison isolates the effect of ephemeral augmentation against the non-ephemeral alternative.

The website presents one scenario that represents a bounded productivity task within the application: a short presentation-refinement exercise foregrounding refinement support.

Participants receive a small slide deck whose order has been intentionally scrambled and must restore it to a coherent narrative sequence before submitting.

The interaction is deliberately narrow:

the main slide canvas functions as a large read-only preview, while the participant performs the task by reordering the slide thumbnails.

The lifecycle of trigger, instantiation, interaction, and

dismissal is implemented as the common interaction logic for both interface conditions.

In the ephemeral condition, temporary support may propose order cues, comparisons, or lightweight explanations of narrative flow, but it does not edit slide content or take over the task.

Success is therefore defined as submitting the deck in the predefined canonical order.

Figure 3 shows the implemented study interface in both conditions.

(a) Baseline condition without temporary support.

(b) Ephemeral condition with temporary support visible.

Figure 3: Screenshots of the implemented experiment interface.

In both conditions, participants

complete the same slide-reordering task: the large slide canvas is a read-only preview, while the actual task action is performed through the thumbnail strip.

The ephemeral condition preserves the same underlying task structure but adds temporary, dismissible support overlays.

The ephemeral condition relies on a constrained support-generation pipeline rather than unrestricted interface generation.

When support is requested or triggered, the system produces a structured interface specification from the current task context, validates it against schema and safety constraints, and either renders it as a temporary overlay or falls back to a deterministic alternative.

The technical details of this pipeline, including the specification model, the validation stages, and the rendering layer, are described in Section 3.

Although the ephemeral condition is allowed to be visually and structurally expressive, that expressiveness remains bounded by the framework.

Temporary support may add task-relevant interface elements, annotate or highlight existing content, expose comparison views, or locally reconfigure a small region of the page.

However, it does not replace the full application, arbitrarily reorganise the entire interface, or perform consequential actions without confirmation.

In practice, the role category, lifecycle, task goals, and success criteria are fixed in advance, while local details such as wording, defaults, and the exact form of low-risk support may vary in response to immediate context.

The ephemeral version is therefore not intended to be wholly

different for every participant.

Instead, it is constrained adaptation : users encounter the same scenario structures and the same framework rules, but the temporary support may be parameterised slightly differently within those limits.

This preserves analytical robustness while still allowing the artifact to reflect the broader motivation of adaptive, goal-relevant ephemeral interfaces.

2.4 Evaluation Design

2.4.1 Experimental Design

The framework will be evaluated through a comparative within-subjects user study using the website artifact described above.

Each participant will complete two trials in total: the same slide-refinement scenario once as Version A and once as Version B.

The presentation order of those versions is fixed, with Version A always shown first and Version B second, but the mapping between version label and experimental condition is randomised per participant so that one participant may receive the baseline first while another receives the ephemeral condition first under the alternative label.

A within-subjects design is appropriate because it allows each participant to serve as their own control, thereby reducing the influence of individual differences in participants' digital literacy, task strategy, and previously acquired familiarity with similar interfaces.

Having version labels preserves a consistent study flow while still varying which version corresponds to assistance across participants.

2.4.2 Participants

Participants will be recruited as regular users of web-based applications who can complete common digital tasks in a desktop browser environment.

Recruitment will follow a convenience sampling approach appropriate to the scope of the capstone project.

Inclusion criteria will focus on participants who are able to use standard web interfaces independently, while any exclusion criteria will be limited to factors that would prevent valid completion of the study tasks.

The final sample size will depend on recruitment feasibility, but the study is designed to support repeated-measures comparisons between the two interface conditions.

2.4.3 Materials and Procedure

The primary study material is the experimental website, which implements both the original and ephemeral interface conditions.

Participants will complete one bounded scenario twice:

a presentation-style refinement task in which they reorder a short slide deck into the intended story sequence. The task structure, goals, and success criteria remain constant for all users so that the study remains comparable across sessions.

Within that shared structure, the ephemeral condition may vary in how temporary support is presented so that the assistance can respond to local context while remaining constrained by predefined design rules derived from the frame- work.

In other words, the study does not test unlimited personalisation or unrestricted interface takeover.

It tests bounded interface adaptation in which temporary support may add, highlight, compare, or locally rearrange elements while preserving interface continuity and user control.

The procedure will be fully web-based.

After opening the study website, participants will read the study information, provide consent, and complete a short background questionnaire about their comfort with websites and familiarity with AI-assisted tools.

The website will then present two task trials in a fixed sequence of Version A followed by Version B, representing the same scenario under the two experimental conditions, with the condition-to-version mapping randomised per participant.

Before each trial, participants will receive a short task prompt that labels the interface version.

During each trial, the website will automatically log behavioural data.

After each trial, participants will complete brief ratings on intrusiveness, perceived control, and helpfulness.

After both trials are complete, participants will answer a final comparative questionnaire about overall preference between the two conditions, with an explicit reminder of which version corresponded to assistance for that participant.

This fully instrumented procedure keeps the study structure consistent while allowing all behavioural and self-reported data to be collected in the same environment.

2.4.4 Measures

The evaluation combines behavioural and self-reported measures in order to capture both task performance and perceived experience.

Behavioural measures are derived from interaction logs recorded by the website, while subjective measures are collected through short post-trial and post-study questionnaires.

The core claim being tested is not simply that ephemeral interfaces are preferred, but that they can improve task support without introducing additional disruption.

To assess that claim, the study will measure efficiency through task completion time, effectiveness through task completion rate, friction through interaction count, and user experience through ratings of intrusiveness, perceived control, helpfulness, and overall preference.

In addition, logs from ephemeral interactions will record whether temporary support was triggered, requested, shown, used, dismissed, ignored, or expanded for further detail, allowing the study to interpret how participants actually engaged with the generated assistance.

Table 1: Dependent measures and collection methods.

Measure	Type	Collection	Analysis
Task completion rate	Binary	Event logging	McNemar's exact test
Task completion time	Continuous	Timestamps	Wilcoxon signed-rank test
Interaction count	Discrete	Event logging	Wilcoxon signed-rank test
Intrusiveness	Ordinal (1-7)	Post-trial Likert item	Wilcoxon signed-rank test
Perceived control	Ordinal (1-7)	Post-trial Likert item	Wilcoxon signed-rank test
Helpfulness	Ordinal (1-7)	Post-trial Likert item	Wilcoxon signed-rank test
Preference	Nominal	Final forced choice	Binomial test

2.4.5 Data Collection

Data collection will combine instrumented event logging with questionnaire responses.

For each trial, the website will record task start and end times, completion outcomes, interaction counts, submitted task state, and the active interface condition.

In the ephemeral condition, logs from ephemeral interactions will capture support trigger events, explicit requests, support shown events, dismissals, usage confirmations, ignored support, detail expansions, and the generated support outputs themselves.

Post-trial survey items will capture perceived intrusiveness, perceived control, and helpfulness, while the final questionnaire will capture overall preference between conditions.

To link behavioural and survey data, each session is associated with a cookie-linked participant identifier that is stored consistently across the website logs and questionnaire responses.

No names or email addresses are collected; the study stores only age range, occupation, familiarity ratings, trial logs, and questionnaire responses.

2.4.6 Analysis Plan

The primary analysis will compare the original and ephemeral conditions on each dependent measure using paired statistical tests selected for the data type, scale of measurement, and expected sample size.

Task completion rate is a paired binary outcome: each participant either completes the task correctly or does not, under each condition.

This will be analysed using McNemar's exact test, which evaluates whether the proportion of participants who change completion status between conditions differs from what would be expected by chance.

Completion proportions for each condition will be reported alongside the test result.

Task completion time and interaction count are continuous and discrete measures, respectively, that are paired across conditions.

Because the expected sample size is modest and time data is typically right-skewed, both measures will be analysed using the Wilcoxon signed-rank test, a non-parametric alternative to the paired t-test that does not assume normality of the paired differences.

Medians and interquartile ranges will be reported for each condition.

Post-trial Likert ratings for intrusiveness, perceived control, and helpfulness are ordinal measures on a seven-point scale.

Each rating will be analysed using the Wilcoxon signed-rank test, which is appropriate for paired ordinal data.

Medians will be reported rather than means to respect the ordinal nature of the scale.

Overall preference is a forced-choice measure in which each participant selects which condition they preferred. This will be analysed using a binomial test against the null hypothesis that preference is split equally between conditions.

The count and proportion choosing each condition will be reported.

Ephemeral-only interaction logs will be analysed descriptively to show the proportion of participants for whom support was triggered, shown, used, dismissed, ignored, or expanded, which helps interpret whether a null or positive effect reflects the quality of the support itself or low engagement.

The analysis will therefore focus on whether the ephemeral condition yields improvements in efficiency, effectiveness, or perceived helpfulness without increasing intrusiveness or reducing perceived control.

Because the ephemeral experience may vary within bounded framework rules, the analysis treats that variation as part of the intended adaptive

behaviour of the artifact rather than as an uncontrolled change to the task itself.

3 Implementation

The previous section defined the conceptual framework, the artifact's purpose as a research instrument, and the evaluation design that structures the comparative study.

This section describes how those ideas were translated into a working system.

The goal is not to document every engineering decision, but to make the technical design legible enough that the reader can assess whether the artifact faithfully operationalises the framework and whether the instrumentation is sufficient to support the evaluation.

3.1 Architecture Overview

The artifact is built as a full-stack web application using the Next.js App Router with TypeScript (Vercel, 2026b). The server layer exposes a set of API routes that handle participant management, trial orchestration, event logging, questionnaire storage, and ephemeral support generation.

Persistent data is stored in a PostgreSQL database.

The ephemeral support system relies on Google Gemini, accessed through the Vercel AI SDK, to produce structured interface specifications from the current task state (Google, 2026; Vercel, 2026a).

These choices reflect the requirements of a research instrument rather than a production application.

Next.js (Vercel, 2026b) was selected because its App Router collocates server-side logic and client rendering in the same project, which simplifies the instrumentation of both request handling and user interaction.

TypeScript and Zod provide compile-time and runtime type safety across the generation pipeline, which is critical when validating AI-generated output against a constrained specification.

PostgreSQL and Drizzle offer relational integrity and migration tooling appropriate for structured experimental data.

The Vercel AI SDK (Vercel, 2026a) abstracts provider-specific details while supporting structured output modes, which allows the generation step to request JSON that conforms to a predefined schema rather than parsing free-form text.

Figure 4: High-level system architecture of the experimental artifact.

3.2 Participant Flow and Session Management

The study procedure described in the methodology is implemented as a linear state machine that determines which page the participant should see at any point during the session.

When a participant first opens the website, they are presented with the study information and consent form.

Providing consent creates a participant record in the database and sets an HTTP-only session cookie that associates all subsequent requests with that record.

No traditional authentication is required because the study collects no directly identifying personal data; the cookie serves only to link a browser session to its corresponding participant row.

After consenting, the participant completes a short background questionnaire covering age range, occupation, familiarity with web applications, and familiarity with AI-assisted tools.

Submitting the background questionnaire acknowledges the study instructions and triggers the creation of the first trial row.

From this point forward, a server-side state resolution function inspects the participant's record, their trial rows, and their questionnaire responses to determine the current study step.

The possible steps form a fixed sequence: background , study (an active trial), post-trial questionnaire , final questionnaire , and complete .

The client-side routing func-

tion maps each step to the corresponding page, so that refreshing the browser or navigating away always returns the participant to the correct point in the procedure.

Trial rows are created incrementally rather than all at once.

The first trial is created when the participant acknowledges the instructions.

Each subsequent trial is created only after the previous trial has been completed and its post-trial questionnaire has been submitted.

This ensures that the participant cannot skip ahead or access later trials before finishing earlier ones.

Counterbalancing is implemented at the participant level.

When a participant record is created, a boolean flag is randomly set to determine whether the baseline condition corresponds to Version A or Version B.

The trial plan then assigns conditions to version labels using this flag, so that one participant may encounter the baseline first while another encounters the ephemeral condition first, both under the neutral label "Version A."

3.3 Data Model

The database schema consists of five tables designed to capture the behavioural and self- reported data required by the evaluation measures defined in the methodology.

Figure 5 shows the entity-relationship diagram (ERD): each participant may complete several trials; each trial may emit many logged events and many stored ephemeral support generations; questionnaire responses belong to a participant, and may optionally reference a trial (post-trial items) or omit a trial key (final questionnaire).

Table 2 summarises what each table contributes to the evalua- tion.

The schema preserves referential integrity and supports the study queries needed for export and analysis.

3.4 Task Scenario Implementation

The study evaluates one scenario, the slide-reordering task described in the methodology.

The implementation uses a scenario registry where each scenario's metadata is defined, including its taxonomy category, correct answer, allowed ephemeral targets, task prompts, and a function

1:N participants 1:N questionnaire responses trials * 1:N 1:N 1:N support outputs trial events

Figure 5: Entity-relationship diagram of the study database.

Directed edges are labelled with

cardinalities: one participant to many trials; one trial to many events and many support records; one participant to many questionnaire rows.

The dashed edge indicates an optional foreign key:

trial id is NULL for final-questionnaire rows.

that builds the initial task state.

This registry-based design means that adding a new scenario requires only a new registry entry rather than changes to the study flow or state machine.

The slide-reordering scenario presents participants with a short deck of four slides whose order has been intentionally scrambled. The main canvas area displays a read-only preview of the currently selected slide, while the actual task interaction occurs through a horizontal strip of draggable thumbnail cards. Drag-and-drop is implemented using the @dnd-kit library, which provides accessible keyboard and pointer-based reordering with sortable constraints.

Participants reorder the strip and submit when they believe the sequence is correct.

Correctness is evaluated by comparing the submitted slide order against a predefined canonical order. Each scenario variant defines its own canonical sequence: variant A follows a corporate readout structure (title, problem, metrics, ask), while variant B follows a committee funding structure (meeting frame, policy, proof, ballot items). The variant assignment uses a deterministic hash of the participant identifier and scenario identifier to assign variant A or variant B to the baseline condition, with the opposite variant assigned to the ephemeral condition.

This ensures that each participant encounters two different slide decks across the two trials, preventing familiarity effects from the task content while keeping the task structure identical.

Both conditions share the same task component, the same interaction mechanics, and the same submission logic. The only difference is whether the ephemeral support layer is active.

Table 2: Summary of each table's role relative to the evaluation measures.

Table	Role in the study
participants	Consent and background questionnaire; counterbalancing flag (baseline is version a); links all rows for a session.
trials	One row per task trial: condition, scenario, timing, correctness, submitted answer, interaction count-supports performance and efficiency measures.
trial events	Typed events with JSON payloads-supports interaction counts and ephemeral engagement (e.g. questionnaire responses
	Post-trial Likert items and final preference; trial id set for post-trial rows, null for final rows-supports subjective mea- sures.
support outputs	Ephemeral condition only: model input state and output spec per generation-supports analysis of support quality and fallback.

This design isolates the effect of ephemeral augmentation from other variables.

3.5 Ephemeral Support System

The ephemeral support system is the core technical contribution of the artifact. It translates the framework's concepts of taxonomy, lifecycle, and design principles into a constrained generative pipeline that produces temporary interface overlays without replacing the host application.

This subsection describes the three layers of the system: the specification model that defines what can be generated, the pipeline that produces and validates specifications, and the rendering layer that displays them.

3.5.1 Component Specification Model

Ephemeral support is represented as a typed tree structure called an EphemeralSpec . Each specification contains a version number, a root node, and metadata indicating whether the over- lay is dismissible. Nodes are recursive: each node has a type, a property bag validated against a per-type schema, and optional children. The tree is constrained to a maximum depth of four levels and a maximum of six children per node. The decision to use JSON as the intermediate structure for these specifications was inspired by JSON Render, which similarly demonstrates how interface descriptions can be expressed declaratively and then rendered from structured data (Jason Lengstorf, 2026).

Table 3: Ephemeral component catalog.
Each type is validated against a per-component Zod

schema that enforces field types, string lengths, and structural constraints.

Category	Component type	Role	Visual cues
Visual cue components	HighlightRing	Amber ring around a target element	PulseRing
	Animated ring with configurable duration	ArrowCue	Arrow pointing at a target
	FocusMask	Dims everything except the target	Anchored annotations
	AnchoredTooltip	Plain-text tooltip near a target	HintStack
	Ordered list of short hints near a target	InspectPanel	Expandable panel with summary and optional de- tail lines
Anchored annotation components	ConsequenceNote	Single italic consequence line near a target	SideNote
	Margin annotation beside a target	Relational	ConnectorLine
	Dashed line between two targets	ComparisonStrip	Short contrast text positioned under two targets
Relational components	StepRail	Numbered callouts across two to six targets	Layout containers
	Stack	Vertical sequence container	ViewportPanel
	Absolutely positioned panel in viewport percent- ages	TargetOffsetPanel	Panel positioned relative to a target with pixel off- sets
Layout containers	Rich content	FlowHtml	Sanitised HTML block (only inside ViewportPanel)
	AnchoredHtml	Sanitised HTML anchored near a target	The component catalog defines seventeen component types, organised into five functional categories.

Table 3 lists the types and their roles.

Visual cue components draw attention to specific interface elements without adding content.

Anchored annotation components attach textual or rich content near a target element.

Relational components connect or compare two or more targets to support reasoning across elements.

Layout containers position children within the viewport or relative to a target.

Rich content components render sanitised HTML and inline SVG for more expressive explanations such as flow diagrams or annotated illustrations.

Each component type is associated with a Zod schema that validates its properties at run-time. These schemas enforce constraints such as maximum string lengths, numeric ranges, and required fields. For example, an `InspectPanel` requires a target identifier, a title of at most

120 characters, a summary of at most 280 characters, and optionally up to four detail lines of

at most 200 characters each.

These limits prevent the model from generating excessively long or unbounded output while leaving enough room for contextually relevant content.

The catalog is versioned so that changes to the available component types or their constraints can be tracked across study sessions.

3.5.2 Generation and Validation Pipeline

When ephemeral support is requested during a trial, the system executes a multi-stage pipeline that generates a specification, validates it against the catalog and scenario constraints, and either accepts it or falls back to a deterministic alternative.

Figure 6 illustrates this pipeline.

The generation step constructs two prompts.

The system prompt describes the specification schema, lists all available component types with their property schemas, states the allowed target identifiers for the current scenario, provides taxonomy-specific guidance (such as which components are most appropriate for refinement support), and enumerates the design rules that constrain output structure.

The user prompt includes the current task state, including slide order, deck metadata, and review goals, together with an optional participant interaction snapshot that reflects the participant's progress at the moment support was requested.

This snapshot allows the model to tailor its output to what the participant has already done rather than generating generic first-visit guidance.

The system sends these prompts to Google Gemini using the Vercel AI SDK's structured output mode (Google, 2026; Vercel, 2026a), which constrains the model to produce JSON conforming to a provided Zod schema. The schema used for model output is a simplified subset of the full specification schema, designed to avoid compatibility issues with the provider's structured output implementation while remaining strict enough to prevent most structural errors at generation time.

The model's output then passes through a series of server-side validation stages:

1. Envelope parsing.

The top-level structure is checked for a valid version number, root node, and metadata.

2. Node tree parsing.

The root and its descendants are recursively validated for correct

node shape, child count limits, and maximum depth.

3. Per-component property validation.

Each node's properties are validated against its

type-specific Zod schema, enforcing field types, string lengths, and numeric ranges.

4. Target allowlisting.

Every target identifier referenced in the specification is checked

against the scenario's predefined list of allowed targets.

Any reference to an interface element not declared in the scenario registry causes the specification to be rejected.

5. Structural container rules.

Layout containers are checked to ensure they contain at

least one child.

This applies to Stack , ViewportPanel , and TargetOffsetPanel .

6. Ancestry constraints.

FlowHtml nodes are permitted only within a ViewportPanel

subtree, preventing free-floating rich HTML from appearing anywhere in the interface.

7. HTML sanitisation.

Any HTML content in FlowHtml , AnchoredHtml , or SideNote

nodes is passed through a server-side sanitisation step using DOMPurify with a strict allowlist of tags and attributes.

This prevents injection of scripts, event handlers, or disallowed elements while permitting semantic HTML and inline SVG for diagrams.

If any stage rejects the output, the system falls back to a deterministic, hardcoded specification for the current scenario.

Fallback specifications are defined per scenario and per variant, ensuring that every participant in the ephemeral condition receives some form of support regardless of model behaviour.

The fallback specifications use the same component types and target identifiers as model-generated output, so they appear visually consistent with the rest of the ephemeral experience.

Whether the specification was model-generated or a fallback, the full input state, model name, and output are persisted to the support outputs table.

This logging enables post-study analysis of generation quality, fallback frequency, and the relationship between input context and output content.

To verify the implementation end-to-end outside the aggregate study analysis, a local support- generation workflow was also conducted on 2026-04-04.

Three consecutive hesitation-triggered requests for the slide-reordering scenario completed successfully and were persisted as support- output records without fallback.

A non-trivial multi-component overlay was generated, vali- dated, saved to PostgreSQL, and rendered successfully.

Exact trace details are provided in Appendix A.3.

2. State checks and assem-

3. Generation request

1. Client trigger

bly The syst

FAQs

What is GPTZero?

GPTZero is the leading AI detector for checking whether a document was written by a large language model such as ChatGPT. GPTZero detects AI on sentence, paragraph, and document level. Our model was trained on a large, diverse corpus of human-written and AI-generated text with support for English, Spanish, French, German, and other languages. To date, GPTZero has served over 17 million users around the world, and works with over 100 organizations in education, hiring, publishing, legal, and more.

When should I use GPTZero?

Our users have seen the use of AI-generated text proliferate into education, certification, hiring and recruitment, social writing platforms, disinformation, and beyond. We've created GPTZero as a tool to highlight the possible use of AI in writing text. In particular, we focus on classifying AI use in prose. Overall, our classifier is intended to be used to flag situations in which a conversation can be started (for example, between educators and students) to drive further inquiry and spread awareness of the risks of using AI in written work.

Does GPTZero only detect ChatGPT outputs?

No, GPTZero works robustly across a range of AI language models, including but not limited to ChatGPT, GPT-5, GPT-4, GPT-3, Gemini, Claude, and AI services based on those models.

What are the limitations of the classifier?

The nature of AI-generated content is changing constantly. As such, these results should not be used to punish students. We recommend educators to use our behind-the-scenes [Writing Reports](#) as part of a holistic assessment of student work. There always exist edge cases with both instances where AI is classified as human, and human is classified as AI. Instead, we recommend educators take approaches that give students the opportunity to demonstrate their understanding in a controlled environment and craft assignments that cannot be solved with AI. Our classifier is not trained to identify AI-generated text after it has been heavily modified after generation (although we estimate this is a minority of the uses for AI-generation at the moment). Currently, our classifier can sometimes flag other machine-generated or highly procedural text as AI-generated, and as such, should be used on more descriptive portions of text.

I'm an educator who has found AI-generated text by my students. What do I do?

Firstly, at GPTZero, we don't believe that any AI detector is perfect. There always exist edge cases with both instances where AI is classified as human, and human is classified as AI. Nonetheless, we recommend that educators can do the following when they get a positive detection: Ask students to demonstrate their understanding in a controlled environment, whether that is through an in-person assessment, or through an editor that can track their edit history (for instance, using our [Writing Reports](#) through Google Docs). Check out our list of [several recommendations](#) on types of assignments that are difficult to solve with AI.

Ask the student if they can produce artifacts of their writing process, whether it is drafts, revision histories, or brainstorming notes. For example, if the editor they used to write the text has an edit history (such as Google Docs), and it was typed out with several edits over a reasonable period of time, it is likely the student work is authentic. You can use GPTZero's Writing Reports to replay the student's writing process, and view signals that indicate the authenticity of the work.

See if there is a history of AI-generated text in the student's work. We recommend looking for a long-term pattern of AI use, as opposed to a single instance, in order to determine whether the student is using AI.